Project Technical Summary

1. System Overview

The system captures raw acoustic vibrations from an engine-mounted **piezo sensor** and combines them with rotation pulses from a **pickup coil** on a **Bajaj CT100** motorcycle. The **ESP32** streams this raw data over a high-baud serial link to a computer running **Python** for live Fast Fourier Transform (FFT) frequency analysis.

2. Brief Schematic Explanation

- **Piezo Amplifier Circuit**
  - **1.6V Active Bias Follower (LM358 Stage B):** Offsets the piezo's alternating current (AC) signal so it sits safely within the ESP32's single-sided positive reading window (0V–3.3V).
  - **Non-Inverting Amplifier (LM358 Stage A):** Configured for a fixed voltage gain of 11.
  - **Capacitive Voltage Divider:** Twin **100nF capacitors** on the input line mathematically cut the incoming signal amplitude in half (÷ 2), dropping the net hardware gain to **5.5x** to prevent massive mechanical forces from clipping the sensor.
  - **Electrical Safety Clamping (1N4148 Diode):** Connected to the ESP32's 3.3V power rail to instantly clamp high-voltage spikes and transient electrical waves caused by engine detonation.
- **RPM Input Interface Circuit**
  - **High-Voltage Clamping:** The 10kΩ resistor limits input current, while dual 1N4003 diodes safely clip harsh positive and negative voltage spikes.
  - **Galvanic Isolation:** The PC817 optocoupler separates the noisy motorcycle electrical system from the ESP32 to prevent damage and interference.
  - **Hardware Noise Filtering:** A 100nF capacitor removes high-frequency ignition static, providing a clean falling-edge trigger for stable RPM calculations.

3. Brief Code Explanation

- **ESP32 Firmware:** Configures the hardware I2S peripheral to continuously capture analog data in the background without CPU overhead, storing chunks into RAM via **Direct Memory Access (DMA)**. A background **Hardware Interrupt** tracks milliseconds between engine turns, attaching a 2-byte RPM tracker onto a **1030-byte binary payload** fired at **500,000 baud**.
- **Python Application:** Decodes the binary packet, separates the raw data streams, strips away the DC offset, and applies a **Hann Window multiplier** to eliminate boundary framing errors. It processes the conditioned sensor arrays through a **Real Fast Fourier Transform (RFFT)** to map active peaks within the 3 kHz to 11 kHz target window. Simultaneously, the application reads the trailing byte payload to calculate the live engine **RPM speed**, dynamically displaying both the real-time frequency spectrum plot and the synchronized engine speed readout on a single, interactive, **auto-scaling** canvas.

4. Major System Difficulties Encountered

- **High-Amplitude Power Ripple:** High-frequency switching noise from the power rail leaked directly into the analog circuit, generating a large, fake noise spikes. This was resolved in hardware by adding a **100nF filtering capacitor** on the op-amp input row to sponge and bleed the power-supply ripple directly to the ground rail.
- **Incorrect frequency measurement caused by sampling-rate calibration:** The ESP32 was configured for a nominal 90 kHz I2S sampling rate, but using 90 kHz in the FFT produced a systematic frequency error. For example, a 5 kHz input was detected as approximately 6.15 kHz. Experimental calibration showed that 72,540 Hz provided much more accurate frequency measurements. The frequency processing bottleneck approximately at **11 kHz**. Frequencies passing this point become degrade and distort, meaning higher spectrum targets require either external digital I2S audio chips or more mathematical code patches.
- **Sampling Jitter and Processing Lag:** Initial testing using standard CPU-driven loops resulted in severe timing instability (jitter) and data backlogs whenever the processor tried to handle background interrupts or serial transmission. This timing variance corrupted the Fast Fourier Transform (FFT) math. The difficulty was completely resolved by bypassing the CPU and offloading the data collection to the ESP32's internal **I2S peripheral engine paired with Direct Memory Access (DMA)** [local]. This hardware-driven upgrade locked the sampling intervals to a jitter-free, allowing data transmission and sensor reading to run smoothly in parallel.